

INFORME DE ANÁLISIS ASINTÓTICO

Taller Práctico — Algoritmos de Ordenamiento | ADA

1. Descripción del Experimento

Se implementaron tres algoritmos clásicos de ordenamiento — **Bubble Sort**, **Insertion Sort** y **Selection Sort** — sin utilizar funciones nativas. Cada algoritmo ordenó los mismos **585 registros de personas** provenientes de una base de datos **SQLite** (tabla *personas*), ordenando por el campo **puntaje_evaluacion** de forma ascendente. El tiempo de carga de la BD fue excluido de la medición.

2. Tabla Comparativa de Tiempos de Ejecución

Algoritmo	Tiempo (ms)	Tiempo (s)	Relación vs. más rápido
Bubble Sort	13.2016	0.013202	2.79x más lento
Insertion Sort	4.7251	0.004725	— (el más rápido)
Selection Sort	7.6164	0.007616	1.61x más lento

Resultado: Insertion Sort fue el más rápido (4.73 ms), seguido de Selection Sort (7.62 ms) y Bubble Sort (13.20 ms).

3. Complejidad Teórica por Algoritmo

Bubble Sort

Caso	Notación	Explicación
Peor caso [Big O]	$O(n^2)$	Lista en orden inverso. Se realizan el máximo de intercambios posibles entre elementos adyacentes.
Caso promedio [Big Theta]	$\Theta(n^2)$	En promedio, se realizan comparaciones de orden cuadrático.
Mejor caso [Big Omega]	$\Omega(n)$	Lista ya ordenada. La optimización detecta que no hubo intercambios y detiene el algoritmo en una pasada.

Insertion Sort

Caso	Notación	Explicación
Peor caso [Big O]	$O(n^2)$	Lista en orden inverso. Cada elemento debe desplazarse hasta el inicio del subarreglo ordenado.
Caso promedio [Big Theta]	$\Theta(n^2)$	Cada elemento se desplaza en promedio $n/4$ posiciones: equivale a $O(n^2)$.
Mejor caso [Big Omega]	$\Omega(n)$	Lista ya ordenada. Solo se realizan $n-1$ comparaciones sin ningún desplazamiento.

Selection Sort

Caso	Notación	Explicación
Peor caso [Big O]	$O(n^2)$	Siempre realiza $n*(n-1)/2$ comparaciones sin importar el estado del arreglo.

Caso promedio [Big Theta]	$\Theta(n^2)$	El número de comparaciones siempre es cuadrático, independiente del estado inicial.
Mejor caso [Big Omega]	$\Omega(n^2)$	IMPORTANTE: ni en el mejor caso mejora. Siempre debe buscar el mínimo en toda la parte no ordenada.

4. Conclusión

¿Por qué Insertion Sort fue más rápido? Los tres algoritmos comparten $O(n^2)$ como cota superior, pero difieren en el número real de operaciones elementales:

- **Insertion Sort** realiza *desplazamientos* (1 asignación por paso) y puede terminar la iteración interna anticipadamente al encontrar la posición correcta.
- **Selection Sort** siempre recorre toda la parte no ordenada buscando el mínimo ($n(n-1)/2$ comparaciones fijas), sin poder detenerse antes.
- **Bubble Sort** realiza *intercambios* que cuestan 3 asignaciones por par, lo que lo hace el más costoso en la práctica aunque tenga la misma cota teórica.

En conclusión, Big O describe el crecimiento asintótico, pero el rendimiento empírico depende de las *constantes ocultas* y el tipo de operación (comparación, intercambio o desplazamiento). Con 585 registros, Insertion Sort superó a los otros dos porque minimiza el trabajo real por iteración.